COMP1531
Software Engineering Fundamentals

Week 7

Exceptions

In this Lecture:

– – –

- Learn to handle program errors gracefully for more robust code.
- Understand how exceptions work in JavaScript.
- Explore how to **raise** and **catch** exceptions effectively.

Dealing With Problems

— — —

The simplest way to deal with problems at run-time...

Just crash !!!

or maybe

Exit with Error !!!

```
1  import prompt from 'prompt-sync';
2  const promptFn = prompt();
3
4  export function gradeFromScore(score: number): String {
5
6    if (score < 0 || score > 100 || Number.isNaN(score)) {
7      console.error('Score must be between 0 and 100');
8      process.exit(1);
9    }
10
11   if (score <= 49){
12     return 'FL';       // 0–49  Fail
13   } else if (score <= 64) {
14     return 'PS';       // 50–64 Pass
15   } else if (score <= 74) {
16     return 'CR';       // 65–74 Credit
17   } else if (score <= 84) {
18     return 'DN';       // 75–84 Distinction
19   } else {
20     return 'HD';        // 85–100 High Distinction
21   }
22 }
23
24 const input = promptFn("Please enter your score: ");
25 console.log(gradeFromScore(parseInt(input)));
```

grade_exit.ts

Dealing With Problems

However, if we throw an exception we start to get into a new territory of programming.

grade_exception1.ts

```ts
import prompt from 'prompt-sync';
const promptFn = prompt();

export function gradeFromScore(score: number): String {

  if (score < 0 || score > 100 || Number.isNaN(score)) {
    throw new Error('Score must be between 0 and 100');
  }

  if (score <= 49){
    return 'FL';       // 0–49  Fail
  } else if (score <= 64) {
    return 'PS';       // 50–64 Pass
  } else if (score <= 74) {
    return 'CR';       // 65–74 Credit
  } else if (score <= 84) {
    return 'DN';       // 75–84 Distinction
  } else {
    return 'HD';        // 85–100 High Distinction
  }
}

try {
    const input = promptFn("Please enter your score: ");
    console.log(gradeFromScore(parseInt(input)));
} catch (err){
    console.log(err.message);
    console.log(err.name);
}
```

Exceptions

An *exception* is an event that disrupts the normal flow of a program - usually caused by an error (e.g., invalid input or missing file).

Purpose:
Exceptions allow your program to **recover gracefully** instead of crashing abruptly.

Example (Grading Program):

```
const score = parseInt(input);
if (isNaN(score)) {
  throw new Error("Invalid score entered!");
}
```

→ The program *throws an exception* when the user enters "abc" instead of a number.

→ You can then **catch** this exception and display a friendly message like: *“Invalid score entered!”*

Exceptions

– – –

- Different languages have different conventions for managing unexpected runtime events.
- Javascript relies on Exceptions for the majority of error handling. Unlike C, which has no exceptions

EAFP: Easier To Ask Forgiveness Than Permission

- A common **JavaScript (and Python) error-handling style**.
- Assume code will work, and use try...catch to recover if something goes wrong.
- Encourages writing cleaner, more confident logic instead of over-checking conditions.

Example (Grading Program):

```
try {
  const score = parseInt(input);
  const grade = gradeFromScore(score);
  console.log("Grade:", grade);
} catch (error) {
  console.error("Error: Invalid score entered.");
}
```

✅ Code assumes input is valid

⚠️ If not, the exception handler “forgives” the error gracefully

EAFP: Easier To Ask Forgiveness Than Permission

– – –

👍 Pros

- Simplifies the main logic flow
- Handles multiple error types with one catch block

👎 Cons

- Makes code non-structured
- Harder to reason what code will be executed.

Look Before You Leap

– – –

LBYL is a convention for avoiding errors in popular languages like C

Unlike EAFP it encourages you to check that something can be done before you do it

Pros:

- Doesn't require exceptions
- Code is structured and therefore easier to reason about

Cons:

- Core logic can be obscured by error checks

LBYL: Example

```
import fs from 'fs';

function readFileSafely(filePath: string): string {
  // LBYL: Check first before acting
  if (!fs.existsSync(filePath)) {
    console.error('File does not exist. Cannot proceed.');
    return '';
  }

  // Safe to proceed since the file exists
  const data = fs.readFileSync(filePath, 'utf-8');
  return data;
}

// Example usage
const fileName = 'notes.txt';
const contents = readFileSafely(fileName);
console.log('File contents:', contents);
```

Exception Examples

This program is good in that it throws an exception, but we aren't handling it.

grade_exception1.ts

```ts
import prompt from 'prompt-sync';
const promptFn = prompt();

export function gradeFromScore(score: number): String {

  if (score < 0 || score > 100 || Number.isNaN(score)) {
    throw new Error('Score must be between 0 and 100');
  }

  if (score <= 49){
    return 'FL';      // 0–49  Fail
  } else if (score <= 64) {
    return 'PS';      // 50–64 Pass
  } else if (score <= 74) {
    return 'CR';      // 65–74 Credit
  } else if (score <= 84) {
    return 'DN';      // 75–84 Distinction
  } else {
    return 'HD';       // 85–100 High Distinction
  }
}

try {
    const input = promptFn("Please enter your score: ");
    console.log(gradeFromScore(parseInt(input)));
} catch (err){
    console.log(err.message);
    console.log(err.name);
}
```

Exception Examples

This program is good in that it throws an exception, but we aren't handling it.

grade_exception2.ts

```
import prompt from 'prompt-sync';
const promptFn = prompt();

export function gradeFromScore(score: number): String {

  if (score < 0 || score > 100 || Number.isNaN(score)) {
    throw new Error('Score must be between 0 and 100');
  }

  if (score <= 49){
    return 'FL';      // 0–49  Fail
  } else if (score <= 64) {
    return 'PS';      // 50–64 Pass
  } else if (score <= 74) {
    return 'CR';      // 65–74 Credit
  } else if (score <= 84) {
    return 'DN';      // 75–84 Distinction
  } else {
    return 'HD';       // 85–100 High Distinction
  }
}

try {
    const input = promptFn("Please enter your score: ");
    console.log(gradeFromScore(parseInt(input)));
} catch (err){
    console.log(err.message);
    console.log(err.name);
    const input = promptFn("Please enter your score: ");
    console.log(gradeFromScore(parseInt(input)));
}
```

Exception Examples

Or we could make this even more robust.

grade_exception3.ts

```
import prompt from 'prompt-sync';
const promptFn = prompt();

export function gradeFromScore(score: number): String {

  if (score < 0 || score > 100 || Number.isNaN(score)) {
    throw new Error('Score must be between 0 and 100');
  }

  if (score <= 49){
    return 'FL';      // 0–49  Fail
  } else if (score <= 64) {
    return 'PS';      // 50–64 Pass
  } else if (score <= 74) {
    return 'CR';      // 65–74 Credit
  } else if (score <= 84) {
    return 'DN';      // 75–84 Distinction
  } else {
    return 'HD';       // 85–100 High Distinction
  }
}

let success = false;
while (!success){
    try {
        const input = promptFn("Please enter your score: ");
        console.log(gradeFromScore(parseInt(input)));
        success = true;
    } catch (err){
        console.log(err.message);
    }
}
```

Testing With Exceptions

We can use jests function `.toThrow()` to test if functions are appropriately throwing exceptions.

```
import { gradeFromScore } from "./grades_exception1";

describe('Grade Correctness', () => {

    test('Deals with Valid Scores', () => {
        expect(gradeFromScore(100)).toEqual("HD");
        expect(gradeFromScore(83)).toEqual("DN");
        expect(gradeFromScore(72)).toEqual("CR");
        expect(gradeFromScore(64)).toEqual("PS");
        expect(gradeFromScore(49)).toEqual("FL");
        expect(gradeFromScore(0)).toEqual("FL");
    })

    test('Deals with Invalid Score', () => {
        expect (() => gradeFromScore (-1)).toThrow('Score must be between 0 and 100');
        expect (() => gradeFromScore (120)).toThrow('Score must be between 0 and 100');
    })
```

catch.test.ts

Finally, Types of Errors in Programming

– – –

1. Compile-Time Errors

- Occur **before the program runs** — detected by the **compiler or interpreter**.
- Usually due to **syntax mistakes** or **type mismatches**.
- ❌ The program won't start until fixed.

```
let score: number = "A+";
// ❌ Type error: string is not assignable to number
```

Types of Errors in Programming

---

2. Runtime Errors

- Happen **while the program is running**.
- The code compiles fine, but something goes wrong during execution (e.g., invalid input, file not found, division by zero).
- Can be caught using `try...catch`.

```
try {
  const scores = [85, 90];
  console.log(scores[5].toFixed(2));
} catch (err) {
  console.error("Runtime Error:", err.message);
}
```

Types of Errors in Programming

---

3. Logic Errors

- The program **runs without crashing**, but produces **incorrect results**.
- Hardest to detect — caused by mistakes in algorithm or logic.

Example (Grading System):

```
const scores = [85, 90];
console.log(scores[5].toFixed(2));
// ❌ TypeError: Cannot read properties of undefined



function grade(score) {
  if (score < 50) return "Pass";   // ❌ Wrong logic
  else return "Fail";
}
console.log(grade(80)); // Output: "Fail" (incorrect)
```

Summary

Error Type	When Detected	Example Cause	Fix
Compile-time	Before execution	Syntax / Type issues	Correct the code
Runtime	During execution	Invalid input / Missing file	Handle with `try...catch`
Logic	After execution	Faulty algorithm	Review program logic

Feedback

COMP1531 Feedback Survey